

Substrate Project Software Guide

0. Goals

All new scripts in this project should adhere to the following goals:

- G1.** Outputs must be cleanly separated from source code.
- G2.** Every run must save all data required for full reproducibility, including:
 - parameters,
 - random seeds,
 - numerical results,
 - raw data arrays used in diagnostics or visualization.
- G3.** Naming conventions and directory structure must be consistent across all scripts.
- G4.** Core physics “engines” must be cleanly separated from experiment scripts.

1. Repository Layout

The canonical repository layout is:

```
substrate_demo/  
  engines/          # Core Hamiltonians and substrate rules  
  experiments/      # Scripts that run parameter sweeps and tests  
  configs/          # Optional JSON/YAML run configurations  
  outputs/          # All generated data from experiments  
  analysis/         # Post-processing notebooks and plotting  
  docs/             # Framework notes, guides, theory documents
```

1.1 Engines

Files such as:

```
engines/substrate_scalar_3d.py  
engines/twofermion_3d_engine.py  
engines/threefermion_3d_engine.py
```

Engines:

- Must not write files.
- Provide functions such as `build.hamiltonian(...)` or `run_experiment(params)`.

1.2 Experiments

Files such as:

```
experiments/scalar_defrag_lumps_exp.py
experiments/twofermion_param_regimes_exp.py
experiments/no_signaling_test_exp.py
```

Experiment scripts:

- Parse CLI arguments,
- call engine logic,
- write results to `outputs/`.

2. Naming Conventions

2.1 Script Names

- Engine scripts: `<domain>_<dimension>_engine.py`
- Experiment scripts: `<domain>_<purpose>_exp.py`

2.2 Output Folders

All experiment outputs must be placed in:

```
outputs/<experiment_name>/<run_id>/
```

Where:

- **experiment_name**: script name without `.py`
- **run_id**: timestamp with optional `--tag`

Example directory:

```
outputs/scalar_defrag_lumps_exp/20251124_183045_baseline/
summary.json
params.json
metadata.json
data/
figures/
logs/
```

3. CLI Design

All experiment scripts must accept the following arguments:

- `--output-root` (default: `outputs`)
- `--tag` (string)
- `--seed` (integer)

Scripts should print the parameter summary to both console and log file.

4. Output File Types and Schemas

Each run folder must contain at least:

4.1 Mandatory Files

1. `summary.json`
2. `params.json`
3. `metadata.json`
4. `logs/run.log`

Example `summary.json` schema:

```
{
  "framework_version": "0.1.0",
  "script": "twofermion_param_regimes_exp.py",
  "run_id": "20251124_183045_baseline",
  "timestamp": "2025-11-24T18:30:45",
  "params": { ... },
  "metrics": {
    "E0": -9.10,
    "A_antisym": 0.998,
    "Fs_singlet": 0.997,
    "S_CHSH": 2.828
  },
  "diagnostics": {
    "converged": true,
    "eigsh_iter": 123,
    "warnings": []
  },
  "verdicts": {
    "has_fermionic_core": true,
    "has_bell_violation": true
  }
}
```

4.2 Data Files

Raw arrays should be saved under `data/` using `.npz` or `.npy`.

4.3 Figures

Plots should be saved under `figures/` using `.png` or `.pdf`.

5. Logging and Randomness

5.1 Random Seeds

All randomness must:

- be initialized from `--seed`,
- be recorded in both `params.json` and `metadata.json`.

5.2 Logging

- Console headers must also be written to `logs/run.log`.
- Warnings must appear in both console and `summary.json`.

6. Experiment Script Structure

Every experiment script must implement the following structure:

1. Parse arguments (including `seed/tag/output-root`).
2. Generate a run ID from timestamp + tag.
3. Create output directories.
4. Save parameter and metadata files.
5. Run the engine.
6. Save:
 - summary JSON,
 - all arrays to `data/`,
 - any plots to `figures/`.

7. Engine Script Structure

Engine scripts must:

- Contain no file writing,
- Expose a function `run_experiment(params)` returning a results dictionary,
- Be fully deterministic given a seed.

8. Data Hygiene Rules

- No script may write outside `outputs/`.
- No run may overwrite an existing run folder.
- All filenames must be descriptive and deterministic.
- Failed runs must still produce:
 - `metadata.json`
 - `logs/run.log`

9. Checklist for New Scripts

- Is this an engine or an experiment?
- Does it accept `--output-root`, `--tag`, `--seed`?
- Does it write data only under `outputs/`?
- Does it save `params.json`, `metadata.json`, `summary.json`?
- Are all arrays saved in `data/`?
- Are all plots saved in `figures/`?
- Does the engine avoid all file I/O?
- Are random seeds handled properly?
- Are key metrics and verdicts clearly recorded?